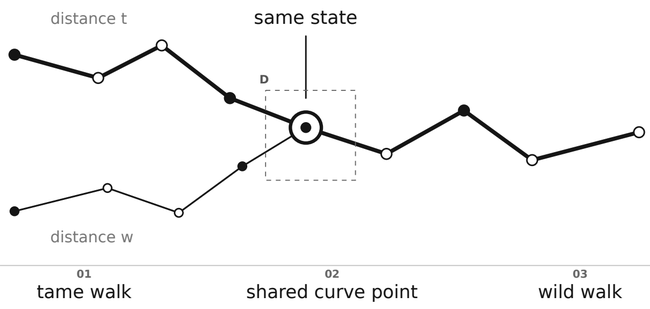
METHODS, DRAWN / ENHANCED

Closer mechanisms, enlarged detail windows, and family-specific monochrome themes

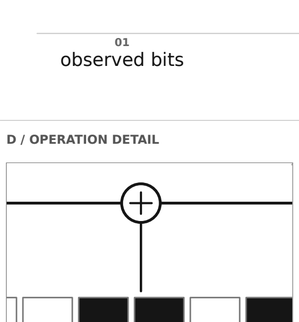
02 / TRACKS / COLLISIONS

M022

Tame/wild collision recovery



D / OPERATION DETAIL



Vector detail / 1.93× main-view scale

HOW THE OPERATION WORKS

A tame walk starts from a known scalar; a wild walk starts from the target point. Their meeting relates recorded distances and yields a candidate scalar.

READ THE THREE STAGES

01 tame walk
02 shared curve point
03 wild walk

WHAT WAS ACTUALLY DONE

collision.py groups same-x entries, tries tame/wild assignments and derives candidates. Because x alone has two point parities, the final scalar must pass exact public-key verification.

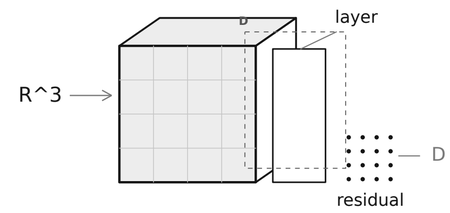
ORIGINAL FINDING / IMPLEMENTED

Same-point opposite-herd distances yield a scalar when the walk origins and distances are valid. No real target collision is established here.

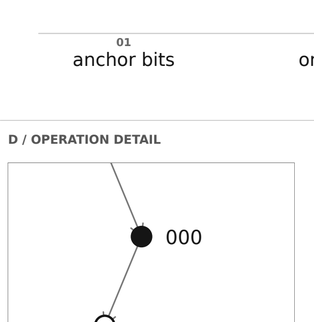
03 / VOLUMES / DECOMPOSITION

M038

Cubic scalar decomposition



D / OPERATION DETAIL



Vector detail / 1.27× main-view scale

HOW THE OPERATION WORKS

Set R1=floor(cuberoot(k)), divide k-R1^3 by R1^2, and retain quotient C1 and remainder D1. This exactly re-encodes k.

READ THE THREE STAGES

01 original scalar
02 cube + layer
03 residual

WHAT WAS ACTUALLY DONE

The code reconstructs known scalars from those digits and checks bounds. For an unknown key the digits are also unknown; an exact identity does not disclose them from the address.

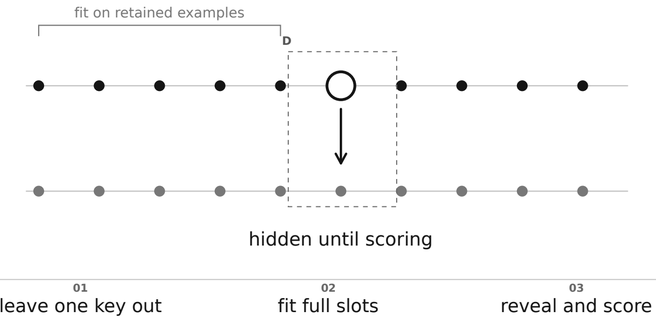
ORIGINAL FINDING / AUDITED/CORRECTED

Exact reconstruction of known integers works; the representation itself supplies no unknown digits or entropy reduction.

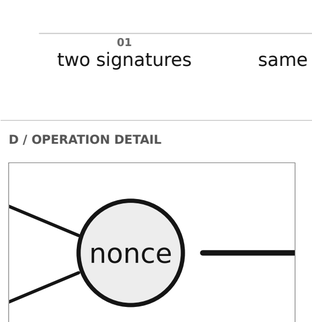
04 / TRACES / VALIDATION

M059

Leave-one-out full-slot backtest



D / OPERATION DETAIL



Vector detail / 1.12× main-view scale

HOW THE OPERATION WORKS

Hide one key from every calibration source, build its full slot list, then reveal the answer only to score containment.

READ THE THREE STAGES

01 leave one key out
02 fit full slots
03 reveal and score

WHAT WAS ACTUALLY DONE

The corrected LOO report separately counts R2, C2, Delta2 and C1 success. Coarse root coverage cannot compensate for a missing exact Delta2 coordinate.

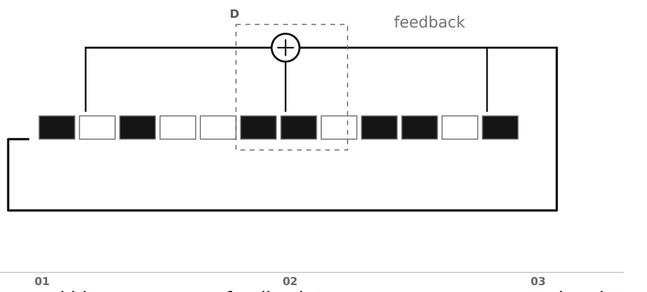
ORIGINAL FINDING / EXECUTED RECORD

Corrected full-slot requirements fail in the documented seven high-index holdouts, despite coarse R2 coverage.

06 / REGISTERS / GENERATORS

M088

LFSR / Berlekamp-Massey reconstruction



D / OPERATION DETAIL

Vector detail / 1.38× main-view scale

HOW THE OPERATION WORKS

An LFSR predicts later bits from a fixed linear combination of earlier bits. Berlekamp-Massey finds a shortest recurrence fitting a supplied bitstream.

READ THE THREE STAGES

01 observed bits
02 feedback taps
03 reconstructed register

WHAT WAS ACTUALLY DONE

Archived generator work applies linear recurrence tests to concatenated known keys. Fitting the observed stream is insufficient unless the same state and output schedule predict unseen keys.

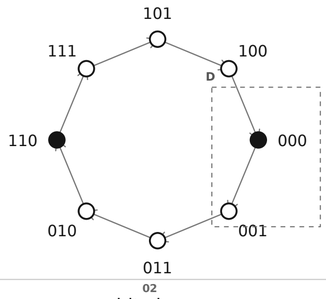
ORIGINAL FINDING / RECORDED RESEARCH

Historical generator review includes bitstream linear-recurrence tests; no transferable unknown-key generator established.

07 / BITS / NEIGHBORHOODS

M108

Gray-code range and anchor walks



D / OPERATION DETAIL

Vector detail / 1.25× main-view scale

HOW THE OPERATION WORKS

Gray code orders bitstrings so adjacent offsets differ in one bit; adding or XORing offsets around an anchor defines a traversal.

READ THE THREE STAGES

01 anchor bits
02 one bit changes
03 Gray-code walk

WHAT WAS ACTUALLY DONE

bit_flip_hunt.py provides range and anchor variants. They visit specified candidates in a different order and do not prove the key is near the selected anchor.

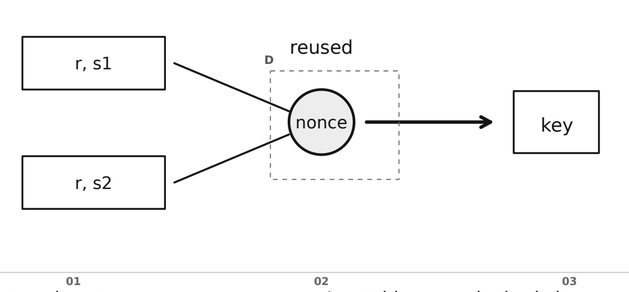
ORIGINAL FINDING / IMPLEMENTED

Enumerate Gray offsets around range floors or metadata anchors; this changes order rather than entropy.

09 / SIGNATURES / CONSTRAINTS

M134

ECDSA nonce-reuse recovery



D / OPERATION DETAIL

Vector detail / 1.60× main-view scale

HOW THE OPERATION WORKS

Reusing the same nonce for different messages creates two equations with the same unknown nonce, allowing elimination and scalar recovery.

READ THE THREE STAGES

01 two signatures
02 same nonce / matching r
03 algebraic key recovery

WHAT WAS ACTUALLY DONE

Synthetic sign/recover tests validate the algebra. Real harvested data are grouped by signer and r; the archived samples did not provide the required exploitable reuse pair.

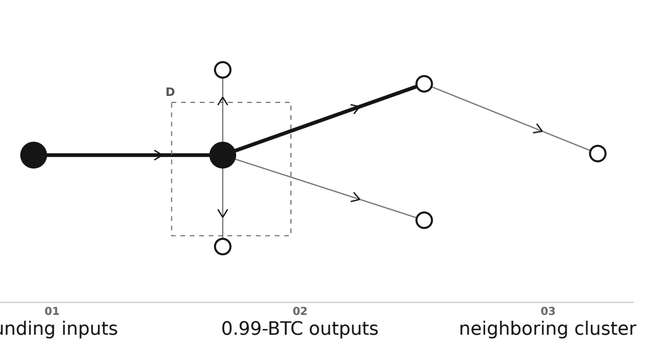
ORIGINAL FINDING / SYNTHETIC/RECORDED TESTS

Algebraic recovery works in planted examples; harvested real signatures showed no matching reuse groups in recorded samples.

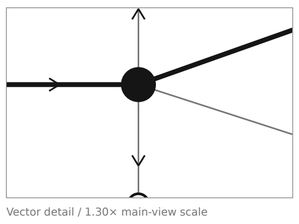
10 / GRAPHS / RELATIONS

M153

0.99-BTC fanout cluster tracing



D / OPERATION DETAIL



Vector detail / 1.30× main-view scale

HOW THE OPERATION WORKS

Detect transactions with repeated 0.99-BTC outputs and traverse their funding/fanout neighbors.

READ THE THREE STAGES

01 funding inputs
02 0.99-BTC outputs
03 neighboring cluster

WHAT WAS ACTUALLY DONE

The crawler uses explicit output-count/value thresholds and bounded BFS expansion. This identifies a transaction pattern or cluster, not an algebraic relation among its private keys.

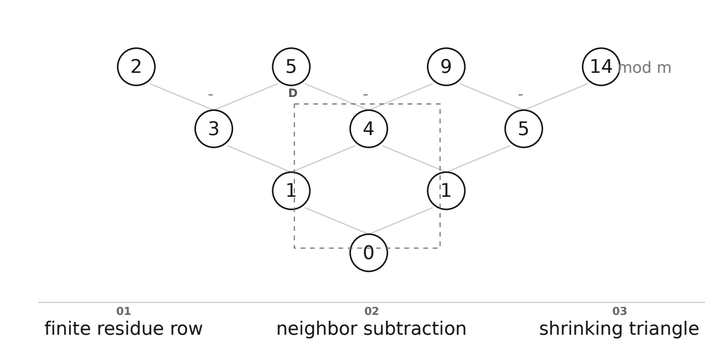
ORIGINAL FINDING / EXECUTED RECORD

Breadth-first funding/output graph searches and message scans are archived.

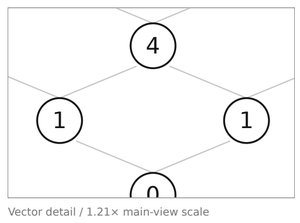
13 / LAYERS / DIFFERENCES

M202

Finite modular difference triangles



D / OPERATION DETAIL



Vector detail / 1.21× main-view scale

HOW THE OPERATION WORKS

Repeatedly subtract neighboring residues modulo m; finite rows shrink by one entry at each level.

READ THE THREE STAGES

01 finite residue row
02 neighbor subtraction
03 shrinking triangle

WHAT WAS ACTUALLY DONE

The experiment follows declared polynomial/exponential residue families through their full finite triangles. Reaching zero on supplied samples does not determine an unknown continuation without a recurrence assumption.

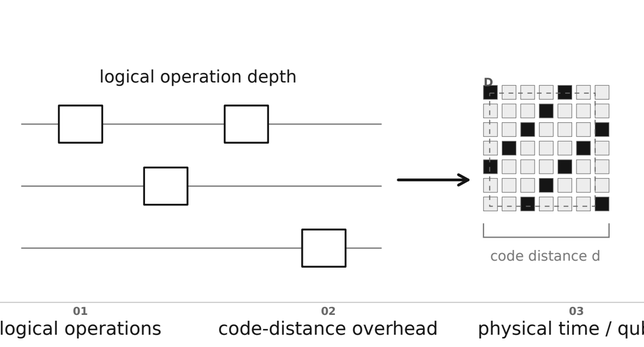
ORIGINAL FINDING / EXECUTED RECORD

Exact finite-row experiments track polynomial/exponential residues through all levels; they do not reconstruct an unknown puzzle key.

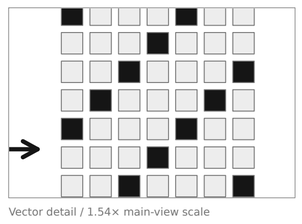
14 / CIRCUITS / RESOURCES

M223

Surface-code/platform timing estimates



D / OPERATION DETAIL



Vector detail / 1.54× main-view scale

HOW THE OPERATION WORKS

Map logical operation counts to physical qubits and runtime using code distance, gate/cycle assumptions and platform parameters.

READ THE THREE STAGES

01 logical operations
02 code-distance overhead
03 physical time / qubits

WHAT WAS ACTUALLY DONE

quantum.py calculates resource estimates for chosen profiles. The outputs are model-dependent arithmetic, not measured throughput on an available quantum computer.

ORIGINAL FINDING / IMPLEMENTED/MODEL

Compare physical errors, distances and cycle times for several platforms; estimates depend on explicit architecture assumptions.